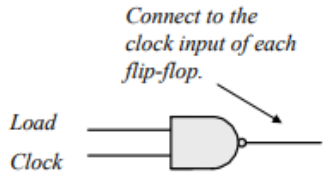
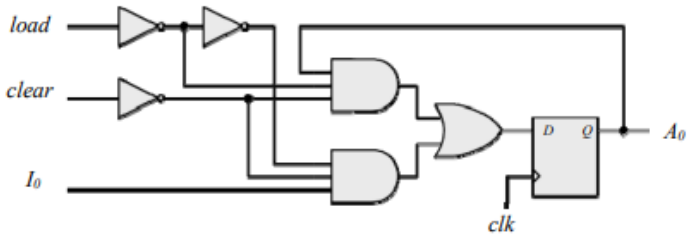
6.1 The structure shown below gates the clock through a nand gate. In practice, the circuit can exhibit two problems if the load signal is asynchronous: (1) the gated clock arrives in the setup interval of the clock of the flip-flop, causing metastability, and (2) the load signal truncates the width of the clock pulse. Additionally, the propagation delay through the nand gate might compromise the synchronicity of the overall circuit.



6.2 Modify Fig. 6.2, with each stage replicating the first stage shown below:



Load	*Clear*	*D*	*Operation*
0	*0*	A_0	*No change*
0	*1*	*0*	*Clear to 0*
1	*x*	I_0	*Load input*

Note: In this design, *clear* has priority over *load*.

6.3 Serial Adder is essentially a sequential circuit that uses shift registers, one full adder and a carry flip-flop. On the other hand, parallel adder is a combinational circuit. Here, *n* full adders are needed to add n-bit binary numbers. A parallel adder gives output in 1 clock cycle, whereas, a serial adder will require *n* clock cycles.

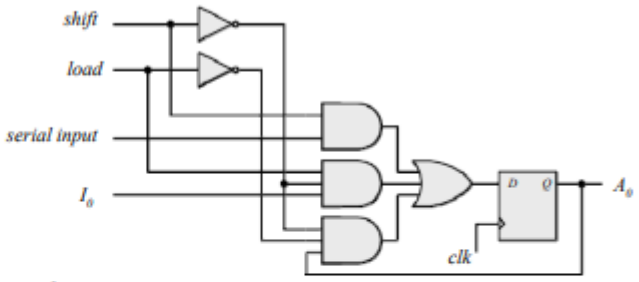
Clearly, the trade-off is complexity in circuitry of parallel adder versus time in serial adder.

6.4 $1010 \Rightarrow 1101 \rightarrow 0110 \rightarrow 0011 \rightarrow 1001 \rightarrow 1100 \rightarrow 0110 \rightarrow 1011$

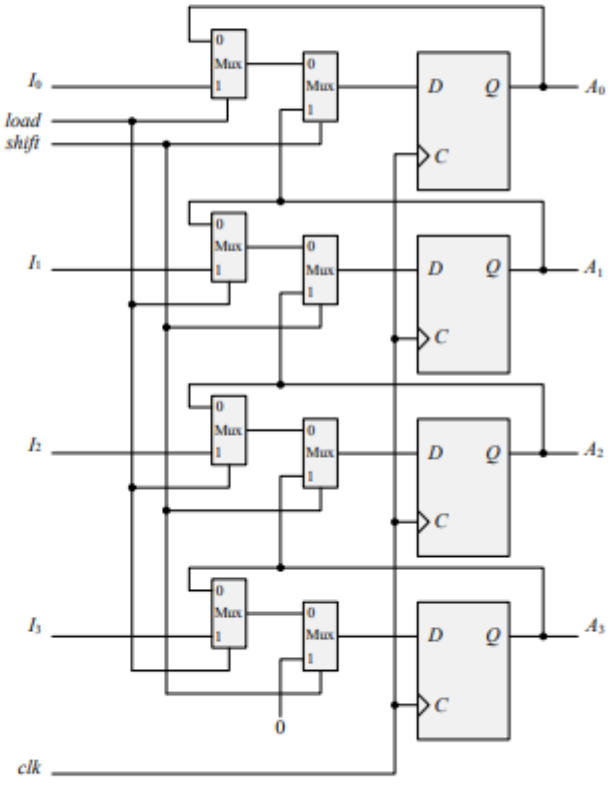
6.5 **(a)** See Fig. 10.12: IC 74194

(b) See Fig. 10.12. Connect two 74194 ICs to form an 8-bit register.

6.6 First stage of register:



Implementation with muxes:

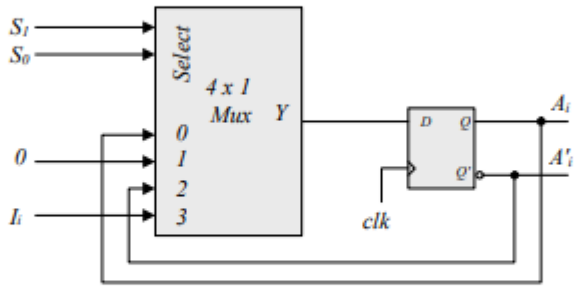


load shift

0	0	No change
0	1	Shift right (Towards LSB)
1	0	Parallel load
1	1	Shift right

6.7 Stage of register: (s1, s0): (0,0) – no change, (1,0) – complement outputs (0,1) – Synchronous clear, (1,1) = Load parallel data.

Stage logic diagram:

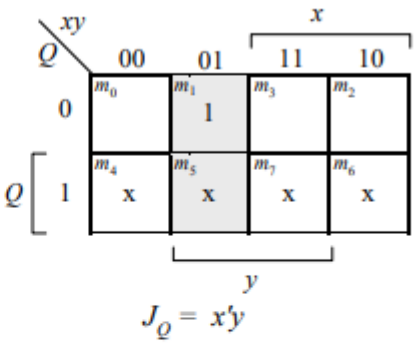


6.8 A = 0010, 0001, 1000, 1100. Carry = 1, 1, 1, 0

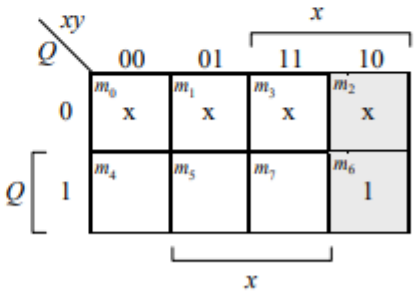
6.9 **(a)** In Fig. 6.5, complement the serial output of shift register B (with an inverter), and set the initial value of the carry to 1.

(b)

Present state Q	Inputs x	y	Next state Q	Output D	FF inputs J_Q	K_Q
0	0	0	0	0	0	x
0	0	0	1	1	1	x
0	0	1	0	1	0	x
0	0	1	0	0	0	x
1	1	0	1	1	x	0
1	1	0	1	0	x	0
1	1	1	0	0	x	1
1	1	1	1	1	x	0



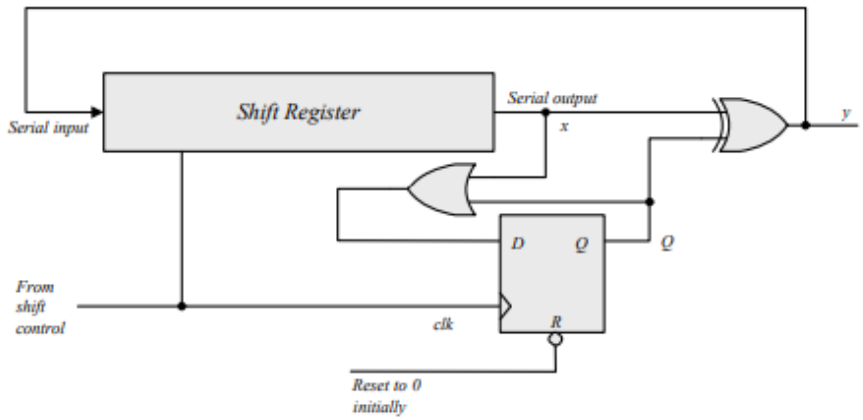
$J_Q = x'y$



$K_Q = xy' \oplus \ \oplus$

$D = Q \oplus x \oplus y$

6.10 See solution to Problem 5.7.
Note that y = x if Q = 0, and y = x' if Q = 1. Q is set on the first 1 from x.
Note that x ⊕ 0 = x, and x ⊕ 1 = x'.



6.11 **(a)** The complement outputs of the flip-flops are connected to the clock input of the next flip-flop.

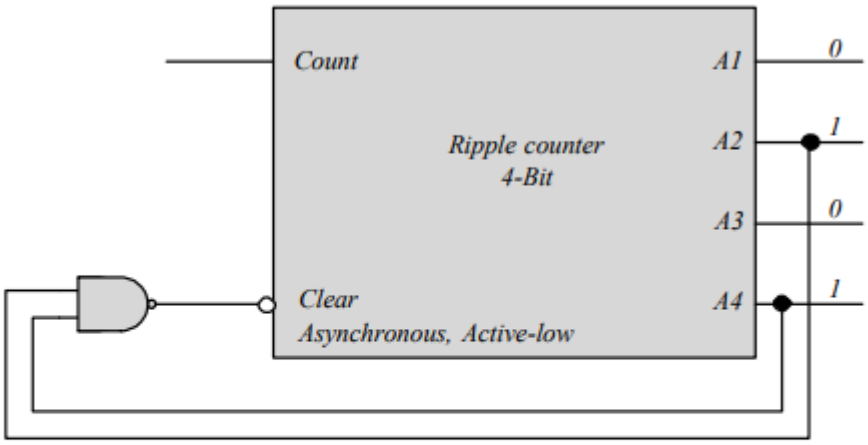
(b) The normal outputs of the flip-flops are connected to the clock input of the next flip-flop.

6.12 Similar to diagram of Fig. 6.8.

(a) With the bubbles in *C* removed (positive-edge).

(b) With complemented flip-flops connected to *C*.

6.13



An active-low asynchronous reset should also be added.

6.14 **(a)** 1100 1101 1**011** → 1100 1101 1**100**; 3 bits
(b) 0000 0**011 1111** → 0000 0**100 0000**; 7 bits
(c) 1110 **1111 1111** → 1111 **0000 0000**; 9 bits

6.15 Maximum delay will be at the worst case when all the 8 flip-flops are complemented.
Maximum Delay = 8 × 4 = 32 ns
Maximum Frequency = 1/t = 1/(32ns) = 10^9/32 ns = 31250000 Hz = 31.25 MHz

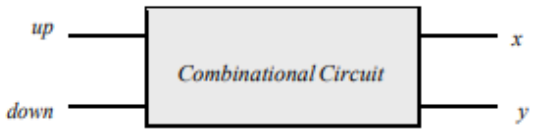
6.16

Q8 Q4 Q2 Q1 :	1010	1100	1110	Self correcting
Next state:	1011	1101	1111	
Next state:	0100	0100	0000	

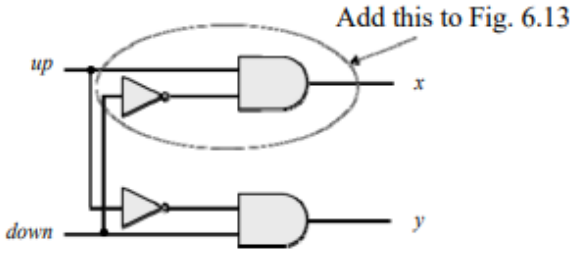
1010 → 1011 → 0100
1100 → 1101 → 0100
1110 → 1111 → 0000

6.17 In the synchronous counter of Fig. 6.12, connect the complements of the flip-flop outputs to the AND gate inputs. Thus, the inputs to *J* and *K* will be:
$J_0 = K_0 = E, J_1 = K_1 = A_0'E, J_2 = K_2 = A_0'A_1'E, J_3 = K_3 = A_0'A'A_2'E$

6.18 When *up* = *down* = 1 the circuit counts up.



up	*down*	*x*	*y*	Operation
0	0	0	0	No change
0	1	0	0	Count down
1	0	1	0	Count up
1	1	0	0	No change



x = *up (down)'*
y = *(up)'down*

6.19 **(b)** From the state table in Table 6.5:

$D_{Q1} = Q'_1$
$D_{Q2} = \sum (1, 2, 5, 6)$
$D_{Q4} = \sum (3, 4, 5, 6)$
$D_{Q8} = \sum (7, 8)$
Don't care: d = $\sum$ (10, 11, 12, 13, 14, 15)

Simplifying with maps:
$D_{Q2} = Q_2Q'_1 + Q'_8Q'_2Q_1$

$D_{Q4} = Q_4Q'_1 + Q4Q'2 + Q'_4Q_2Q_1$
$D_{Q8} = Q_8Q'_1 + Q_4Q_2Q_1$

(a)

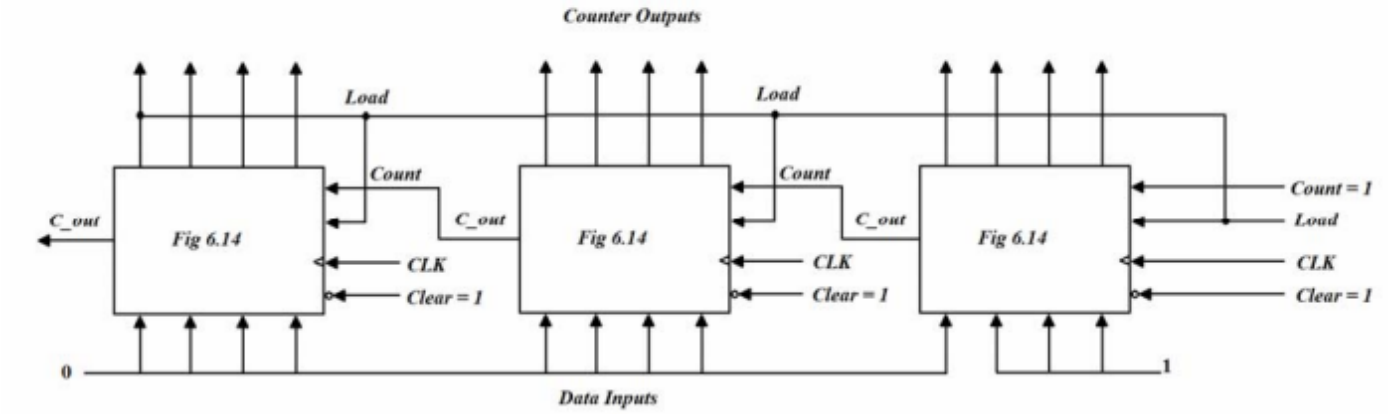
Present state	*Next state*	*Flip-flop inputs*			
$A_8\ A_4\ A_2\ A_1$	$A_8\ A_4\ A_2\ A_1$	$J_{A8}\ K_{A8}$	$J_{A4}\ K_{A4}$	$J_{A2}\ K_{A2}$	$J_{A1}\ K_{A1}$
0 0 0 0	0 0 0 1	0 x	0 x	0 x	1 x
0 0 0 1	0 0 1 0	0 x	0 x	1 x	x 1
0 0 1 0	0 0 1 1	0 x	0 x	x 0	1 x
0 0 1 1	0 1 0 0	0 x	1 x	x 1	x 1
0 1 0 0	0 1 0 1	0 x	x 0	0 x	1 x
0 1 0 1	0 1 1 0	0 x	x 0	1 x	x 1
0 1 1 0	0 1 1 1	0 x	x 0	x 0	1 x
0 1 1 1	1 0 0 0	1 x	x 1	x 1	x 1
1 0 0 0	1 0 0 1	x 0	0 x	0 x	1 x
1 0 0 1	0 0 0 0	x 1	0 x	0 x	x 1

$J_{A1} = 1$
$K_{A1} = 1$
$J_{A2} = A_1A'_8$
$K_{A2} = A_1$
$J_{A4} = A_1A_2$
$K_{A4} = A_1A_2$
$J_{A8} = A_1A_2A_4$
$K_{A8} = A_1$

$d(A_8, A_4, A_2, A_1) = \Sigma\ (10, 11, 12, 13, 14, 15)$

6.20 **(a)** The counter value is reset to 0 each time $A_7A_6 = 11$. So the counter can count from 00000000 to 00111111, i.e. from 0 to 63.

(b) For counting from 7 (0111_2) to 2047 ($0111\ \ 1111\ \ 1111_2$), 3 units are needed. Counter is re-loaded with 0000 0000 0111 when count reaches 2048.



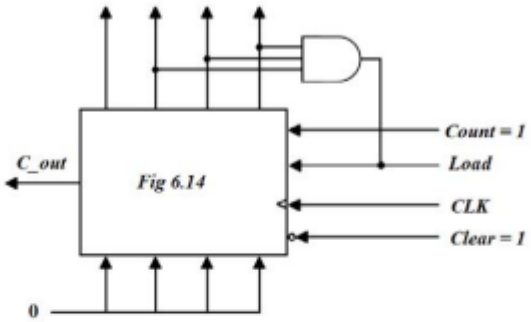
6.21 **(a)**

$$J_{A0} = LI_0 + L'C \qquad KA_0 = LI'_0 + L'C$$

(b)

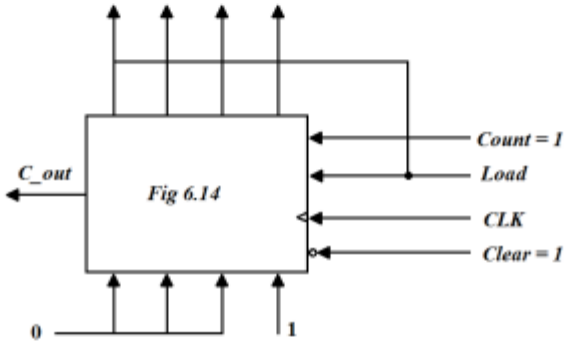
$$J = [L(LI)']'(L + C) = (L' + LI)(L + C)$$
$$LI + L'C + LIC = LI + L'C \text{ (use a map)}$$
$$K = (LI)'\ (L + C) = (L' + I')(L + C) = LI' + L'C$$

6.22 **(a)**



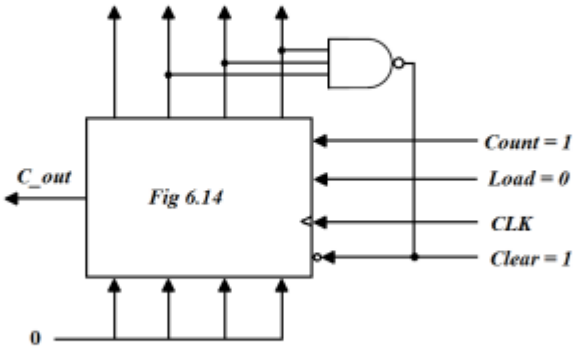
Count Sequence : 0, 1, 2, 3, 4, 5, 6, 7

(b)



Count Sequence : 1, 2, 3, 4, 5, 6, 7, 8

(c)

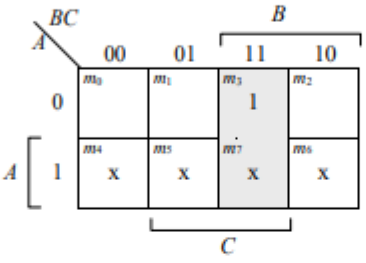


Count Sequence : 0, 1, 2, 3, 4, 5, 6, 7

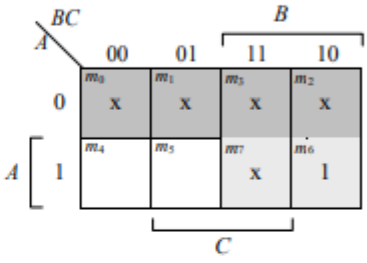
6.23 Use a 4-bit counter and a flip-flop (initially at 0). A start signal sets the flip-flop, which in turn enables the counter. On the count of 11 (binary 1011) reset the flip-flop to 0 to disable the count (with the value of 0000).

6.27

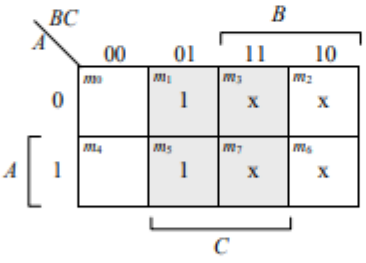
Present state ABC	Next state ABC	Flip-flop inputs J_A	K_A	J_B	K_B	J_C	K_C
000	001	0	x	0	x	1	x
001	010	0	x	1	x	x	1
010	011	0	x	x	0	1	x
011	100	1	x	x	1	x	1
100	101	x	x	0	0	1	x
101	110	x	0	1	x	x	1
110	000	x	1	x	1	0	x
111	xxx	x	x	x	x	x	x



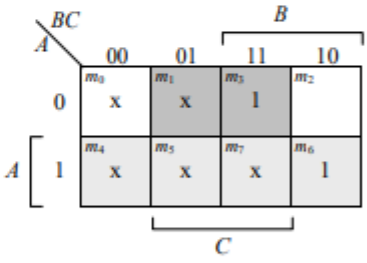
$J_A = BC$



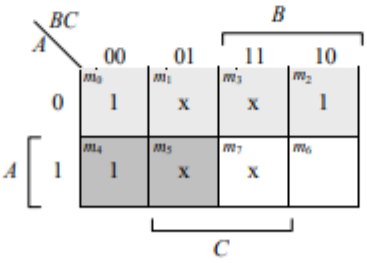
$K_A = A' + B$



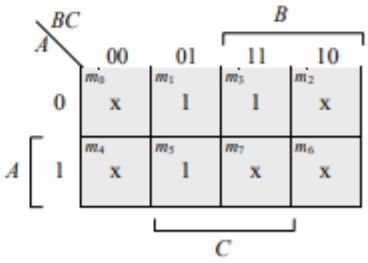
$J_B = C$



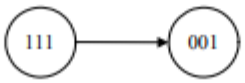
$K_B = A + C$



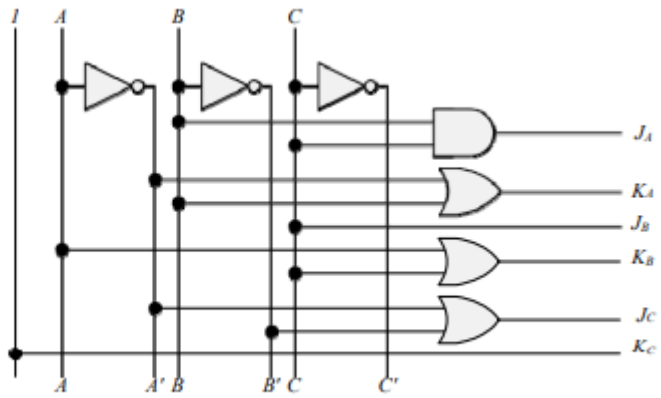
$J_C = A' + B'$



$K_C = 1$

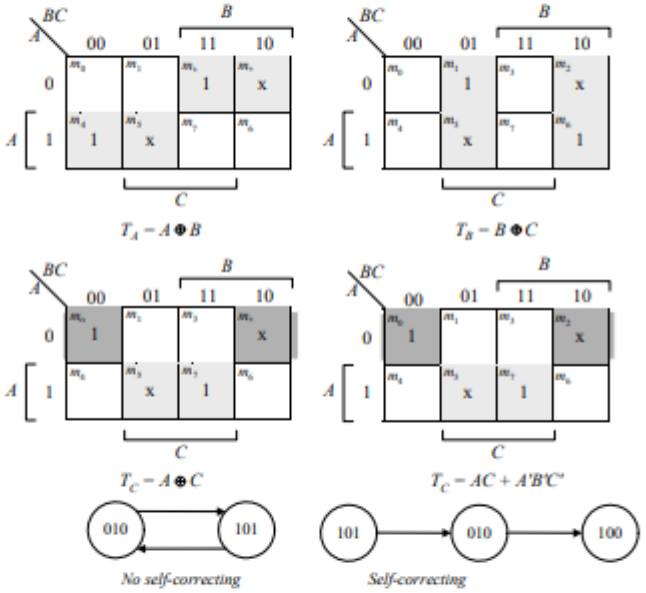


Self-correcting



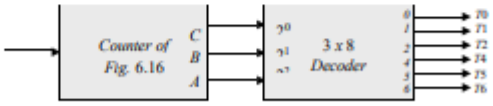
6.24

Present state ABC	Next state ABC	Flip-flop inputs T_A	T_B	T_C
000	001	0	0	1
001	011	0	1	0
010	xxx	x	x	x
011	111	1	1	0
100	000	1	1	0
101	xxx	x	x	x
110	100	0	1	0
111	110	0	0	1

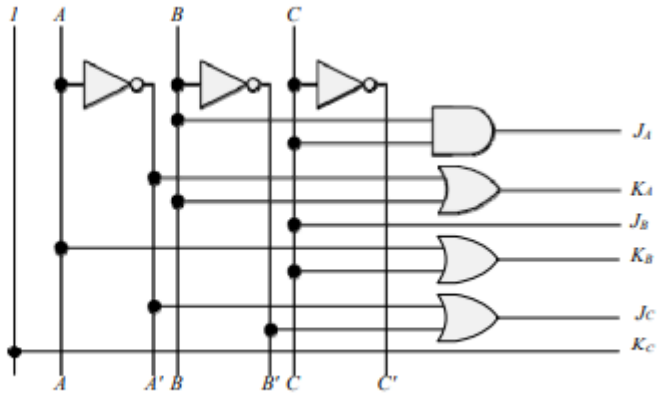


6.25 **(a)** Use a 6-bit ring counter.

(b)

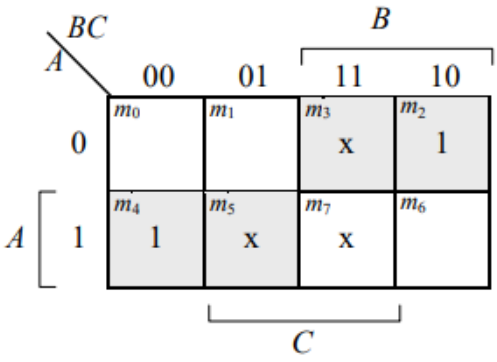


6.26 The clock generator has a period of 8ns. Use a 3-bit counter to count eight pulses.
125/8 = 15.625 MHz; cycle time = $1000 \times 10^{-9}/15.625 = 64$ ns.

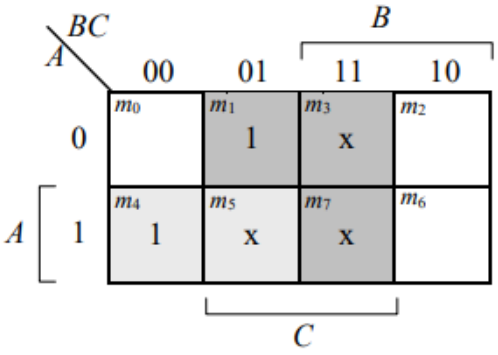


6.28_a, b

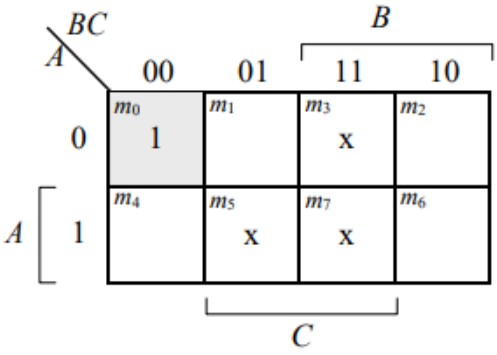
Present state ABC	*Next state ABC*
000	001
001	010
010	100
011	xxx
100	110
101	xxx
110	000
111	xxx



$D_A = A \oplus B$

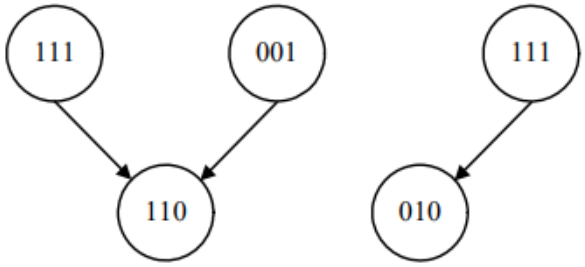


$D_B = AB' + C$



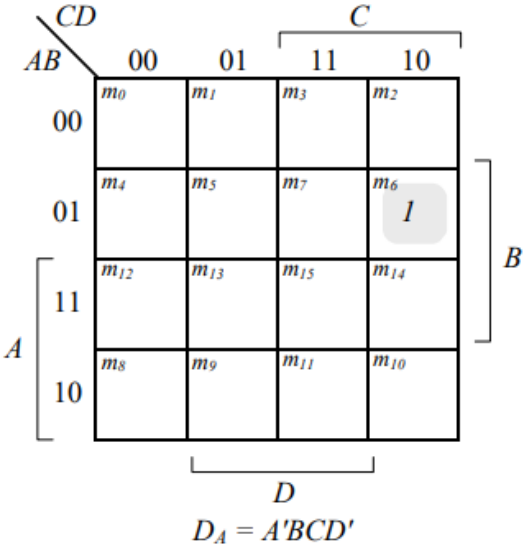
$D_C = A'B'C'$

Self-correcting

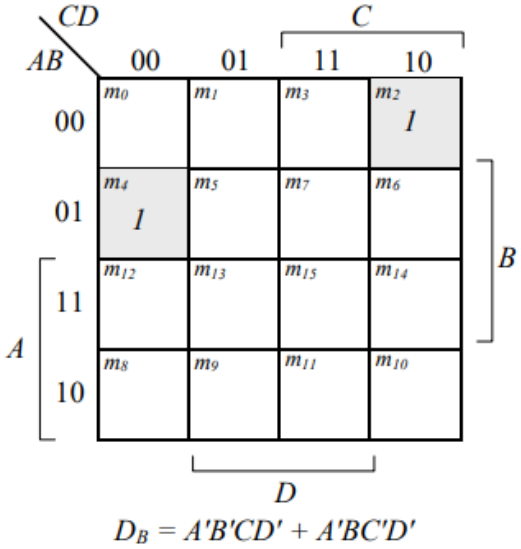


6.28_a, b

Present State	Next State
ABCD	ABCD
0000	0010
0001	0000
0010	0100
0011	0000
0100	0110
0101	0000
0110	1000
0111	0000
1000	0000

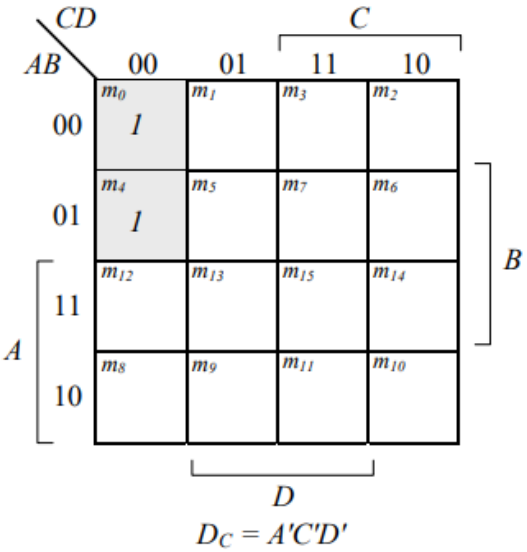


$D_A = A'BCD'$

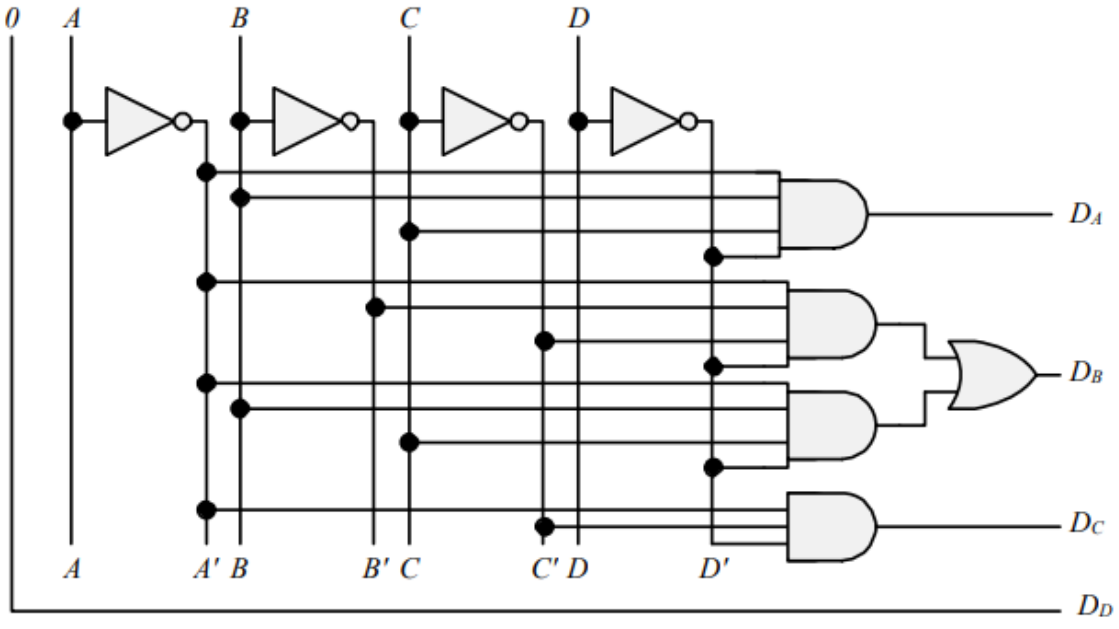


$D_B = A'B'CD' + A'BC'D'$

$D_D = 0$



$D_C = A'C'D'$

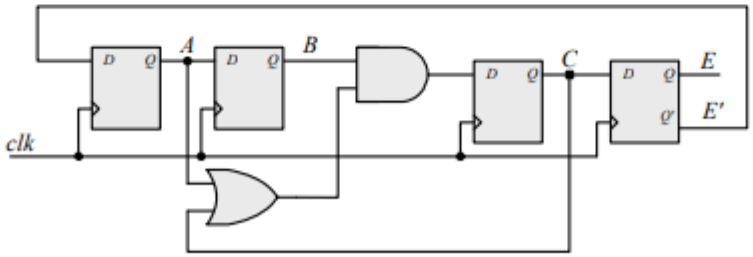


6.29 **(a)** The 8 valid states are listed in Fig. 8.18(b), with the sequence: 0, 8, 12, 14, 15, 7, 3, 1, 0, ...

The 8 unused states and their next states are shown below:

State *ABCE*	*Next state* *ABCE*	*All invalid states*
0000	1001	9
0100	1010	10
0101	0010	2
0110	1011	11
1001	0100	4
1010	1101	13
1011	0101	5
1101	0110	6

(b) Modification: $D_C = (A + C)B$.



The valid states are the same as in (a). The unused states have the following sequences: $2 \rightarrow 9 \rightarrow 4 \rightarrow 8$ and $10 \rightarrow 13 \rightarrow 6 \rightarrow 11 \rightarrow 5 \rightarrow 0$. The final states, 0 and 8, are valid.

6.30 To produce 12 count states, a 6-bit Johnson counter is required that has 6 flip-flops.

Sequence Number	Flip-flop outputs A	B	C	D	E	F	AND gates required for output
1	0	0	0	0	0	0	*A'F'*
2	1	0	0	0	0	0	*AB'*
3	1	1	0	0	0	0	*BC'*
4	1	1	1	0	0	0	*CD'*
5	1	1	1	1	0	0	*DE'*
6	1	1	1	1	1	0	*EF'*
7	1	1	1	1	1	1	*AF*
8	0	1	1	1	1	1	*A'B*
9	0	0	1	1	1	1	*B'C*
10	0	0	0	1	1	1	*C'D*
11	0	0	0	0	1	1	*D'E*
12	0	0	0	0	0	1	*E'F*

Unused states = Total number of states with 6 flip-flops $- 12 = 2^6 - 12 = 52$